

GUÍA DE MODELOS DE IA 2026: la comparativa definitiva

*La guía para elegir el agente que
mejor se adapte a tu proyecto*

Índice

Cómo utilizar esta guía	3
1. Los fundamentos que necesitas para elegir	4
2. Cómo leer Artificial Analysis y LLM Stats	13
3. Los cinco bloques de modelos	16
A. Alta exigencia	17
B. Trabajo diario	18
C. Volumen y presupuesto ajustado	20
D. Pesos abiertos de gran capacidad	22
E. Uso local	24
4. Qué elegir según la tarea que tengas delante	26
5. Entender cuánto cuesta realmente	28
6. Tu propia comparativa: un ejercicio que sí te enseña a elegir	30
7. Errores frecuentes al elegir un modelo	33
8. Llegar al curso con una buena base	34

Cómo utilizar esta guía

El modelo más potente no siempre es el que más te conviene. Para explicar un bucle, revisar una función o transformar datos, puede bastar una opción económica. Para investigar un fallo difícil o modificar un proyecto con muchas dependencias, pagar por más capacidad puede compensar.

La pregunta útil es: **¿qué modelo y qué herramientas me ayudan a resolver esta tarea, con un coste razonable y un resultado que puedo comprobar?**

Esta guía te enseña a responderla. Los nombres cambiarán; los criterios para elegir seguirán siendo útiles.

FUNDAMENTOS + IA + CRITERIO significa comprender el problema, utilizar la IA para avanzar y comprobar que la solución tiene sentido. Si estás aprendiendo a programar, un resultado correcto también debería ayudarte a entender qué has construido.

Qué encontrarás y cómo leerlo

Primero, los fundamentos. Después, una lectura crítica de los rankings y cinco bloques de modelos según su utilidad. Al final encontrarás una tabla para decidir, ejemplos de coste y un ejercicio para comparar modelos con tu propio código.

La selección incluye OpenAI, Anthropic, Google, Meta, xAI, Mistral, Z.ai, Moonshot AI, Alibaba/Qwen y DeepSeek. Es una selección representativa para estudiantes de programación, no un catálogo de todos los modelos existentes. Se centra en lenguaje, código, razonamiento y comprensión multimodal; generar imágenes, vídeo o voz exige valorar también modelos especializados.

Cómo se ha elaborado: las especificaciones y tarifas proceden de documentación oficial o del proveedor de acceso identificado. Artificial Analysis y LLM Stats aportan evaluaciones y contraste. Las recomendaciones de uso son una interpretación práctica de esos datos: no representan pruebas propias ni una garantía de rendimiento.

1. Los fundamentos que necesitas para elegir

1.1. LLM, modelo, aplicación y agente

Un LLM es un modelo de lenguaje de gran tamaño: una red neuronal entrenada para trabajar con secuencias de tokens. Puede generar texto y código, seguir instrucciones y resolver algunas tareas de razonamiento. Que una respuesta resulte convincente no demuestra que sea correcta.

Conviene distinguir cuatro piezas:

Pieza	Qué es	Qué debes comparar
Modelo	El sistema que procesa la entrada y genera una respuesta. Por ejemplo, GPT-6 Astra o Kimi K3.	Capacidad, modalidades, contexto y consumo.
Aplicación	El producto con el que interactúas: un chat, un editor o una herramienta de desarrollo.	Facilidad de uso, funciones, límites y precio del plan.
Proveedor de inferencia	Quien ejecuta el modelo y ofrece acceso, normalmente mediante una API.	Precio, velocidad, disponibilidad y tratamiento de datos.
Agente	Un sistema que combina un modelo con herramientas y un ciclo de acciones y comprobaciones.	Qué puede hacer, con qué permisos y cómo verifica el resultado.

Un chat que escribe «he ejecutado los tests» no demuestra que los haya ejecutado. Necesita una herramienta capaz de hacerlo y un resultado verificable. En una integración mediante llamadas a funciones, el modelo solicita una acción y el software conectado la ejecuta. [Referencia técnica: Llamadas a funciones.](#)

Aplicación práctica: un modelo puede funcionar mejor en una herramienta de programación que en un chat aislado porque recibe los archivos adecuados, ejecuta el proyecto y consulta los errores.

1.2. Parámetros: tamaño no significa inteligencia

Los **parámetros** son valores aprendidos durante el entrenamiento. En las fichas verás abreviaturas inglesas: **B** significa mil millones y **T**, un millón de millones. Un modelo de 8B tiene aproximadamente 8.000 millones de parámetros.

El número ayuda a estimar recursos, aunque no permite ordenar la calidad por sí solo. También influyen la arquitectura, los datos, el entrenamiento y el trabajo de adaptación posterior. No atribuyas a un modelo privado un tamaño que su fabricante no ha publicado.

Hay dos diseños que interesa reconocer:

- **Denso**: se utiliza la mayor parte de los parámetros del modelo en cada paso de generación.
- **MoE (mezcla de expertos)**: se activa una parte de sus componentes para cada token. Se suele distinguir entre parámetros **totales** y **activos**.

La trampa del MoE: pocos parámetros activos pueden reducir el cálculo, aunque no convierten todos los pesos restantes en memoria gratuita. Para alojarlo hay que contar con el conjunto de pesos y con la estrategia de carga del motor de inferencia. La cifra activa no basta para decidir si cabe en tu ordenador. [Explicación de MoE de Hugging Face](#).

1.3. Tokens: la unidad que condiciona contexto y coste

Un **token** es una unidad de representación. En texto puede corresponder a una palabra, un fragmento, un signo o parte de código. La relación entre palabras y tokens varía según el idioma, el contenido y el tokenizador: no uses una equivalencia fija para presupuestar.

En una interacción hay que distinguir:

- **Entrada:** instrucciones, conversación, código y resultados de herramientas que recibe el modelo.
- **Salida:** lo que genera; según el proveedor, la facturación puede incluir también razonamiento que no ves.
- **Entrada en caché:** contenido reutilizado que cumple las condiciones de almacenamiento del proveedor y puede tener una tarifa diferente.

Una conversación larga puede volver a enviar mucho contenido en cada turno. Una respuesta de cinco líneas tampoco implica siempre cinco líneas de consumo. En los modelos de razonamiento de OpenAI, por ejemplo, los tokens internos cuentan como salida facturable. [Tokens y conversación, tokens de razonamiento.](#)

1.4. Ventana de contexto, salida máxima y memoria

La **ventana de contexto** limita la información que el modelo puede manejar en una petición. Según cómo lo documente cada proveedor, verás un límite conjunto o límites separados de entrada y salida. Las instrucciones, los archivos y las herramientas también ocupan espacio; algunos modelos necesitan reservarlo para razonamiento.

La salida máxima es otro límite. Una ventana de aproximadamente un millón de tokens no implica que el modelo pueda escribir un millón de tokens de respuesta.

Tampoco confundas contexto con memoria permanente: una aplicación puede guardar conversaciones o resúmenes, aunque eso es una función del sistema que rodea al modelo.

Más contexto admite más material; no garantiza que el modelo use bien cada detalle. Para un error concreto, empieza por el mensaje de error, el código implicado, las versiones y la forma de reproducirlo. Añade dependencias cuando hagan falta. Puedes comprobar el uso real de documentos largos con evaluaciones específicas como [AA-LCR](#).

1.5. Razonamiento: más esfuerzo tiene un precio

Algunos modelos dedican cálculo adicional a explorar una solución antes o durante la respuesta. Los ajustes pueden llamarse *thinking*, *reasoning effort* o tener nombres propios.

Un esfuerzo mayor puede ayudar en depuración, planificación y problemas con varias restricciones, sin embargo también aumentar los tokens, la espera y el coste. Los nombres de los niveles no están normalizados entre fabricantes.

Recomendación: comienza con un esfuerzo moderado. Súbelo cuando puedas señalar una dificultad concreta o cuando tu evaluación muestre que mejora el resultado. Pedir una respuesta más extensa no equivale a obtener una respuesta más fiable.

1.6. Inteligencia y fiabilidad son cosas distintas

En un ranking, «inteligencia» suele ser el nombre de una puntuación compuesta. Resume el desempeño en determinadas pruebas, con una metodología específica. No mide toda la competencia de un programador ni garantiza que el modelo entienda tu producto.

Para desarrollar software, comprueba al menos cuatro cosas:

- 1. Corrección:** ¿resuelve el problema y los casos límite?
- 2. Seguimiento de instrucciones:** ¿respeta las restricciones y el alcance?
- 3. Honestidad sobre la incertidumbre:** ¿distingue lo comprobado de lo supuesto?
- 4. Verificabilidad:** ¿puedes reproducir y revisar la solución?

Un modelo puede rendir bien en problemas difíciles y equivocarse con una API reciente. Otro puede redactar una explicación clara que contiene un error. Evalúa el resultado con documentación, ejecución y pruebas.

1.7. Multimodalidad: especifica qué entra y qué sale

Un modelo **multimodal** trabaja con más de un tipo de información. Sin embargo, hay que distinguir **entrada** y **salida**.

- Leer una captura puede ayudar a interpretar una interfaz o un error visual.
- Procesar audio o vídeo puede servir para analizar una clase o una grabación.
- Generar una imagen, audio o vídeo es una capacidad distinta de comprenderlos.

Una aplicación puede combinar varios modelos para ofrecer todo eso. En las tablas de esta guía, «texto + imagen → texto» significa que el modelo acepta ambos tipos de entrada y responde con texto. No implica generación nativa de imágenes.

Para cifras y código, facilita el archivo original o el texto cuando lo tengas: una captura puede ocultar datos o introducir errores de lectura.

1.8. Latencia, velocidad y tiempo hasta resolver

Hay tres medidas diferentes:

Medida	Qué te dice	Ejemplo de utilidad
Tiempo hasta el primer token	Cuánto espera el usuario antes de recibir salida.	Conversación y asistencia interactiva.
Velocidad de generación	Cuántos tokens o caracteres salen por segundo.	Respuestas largas y generación de documentos.
Tiempo hasta completar la tarea	Cuánto tarda el flujo entero, incluidas herramientas, reintentos y revisión.	Cambiar código y dejar el proyecto funcionando.

No compares caracteres por segundo con tokens por segundo, ni una medición típica con el percentil de los casos lentos. El proveedor, la carga, el contexto y el esfuerzo de razonamiento cambian la experiencia. [Métricas de Artificial Analysis](#), [ejemplo de métricas por proveedor en LLM Stats](#).

Para programar, mide cuánto tardas en obtener un cambio correcto y revisado. Una respuesta rápida que exige cinco correcciones puede hacerte perder más tiempo.

1.9. Capacidades agénticas: herramientas, permisos y comprobaciones

Para trabajar sobre un proyecto, un agente puede necesitar leer archivos, buscar referencias, editar, usar la terminal, ejecutar pruebas y observar el resultado.

La capacidad útil depende de la combinación **modelo + herramientas + contexto + configuración**. Soportar llamadas a herramientas no demuestra por sí solo que un modelo complete bien proyectos largos.

Como estudiante, empieza por acciones pequeñas y revisables. Utiliza control de versiones y revisa los cambios. Da permisos acordes con la tarea, especialmente si el agente puede publicar, borrar datos o gastar dinero. Un archivo o una web también puede contener instrucciones maliciosas; el sistema debe tratar ese contenido como datos, sin concederle autoridad sobre tu encargo.

1.10. Pesos abiertos, código abierto y modelos privados

Los **pesos** contienen los parámetros aprendidos. Si están publicados puedes, dentro de su licencia, descargarlos y ejecutarlos en infraestructura propia.

Término	Qué permite concluir	Qué no permite concluir
Pesos abiertos	Se distribuyen los pesos bajo una licencia consultable.	Que tengas los datos de entrenamiento, que no haya restricciones o que quepan en tu equipo.
Código abierto	Hay que comprobar qué componentes y condiciones cumplen la definición empleada.	Que publicar pesos baste para abrir todo el sistema de IA.
Pesos privados	El acceso al modelo se ofrece mediante los servicios autorizados por el fabricante.	Que todas sus aplicaciones tengan las mismas funciones o políticas de datos.

La **licencia pertenece a una versión concreta, no a toda una marca**. Una familia puede tener modelos con Apache 2.0 y otros con condiciones propias. La [definición de IA de código abierto de la OSI](#) distingue los pesos de otros componentes necesarios para estudiar y modificar el sistema.

1.11. Local y nube son decisiones de despliegue

En la nube, otro proveedor ejecuta el modelo. Puedes acceder a modelos grandes sin comprar hardware, a cambio de depender de su servicio, sus límites y sus condiciones.

En local, ejecutas el modelo en tu equipo. Ganas control sobre el procesamiento y puedes trabajar sin conexión si el flujo lo permite. Necesitas memoria, un motor compatible y tiempo para configurarlo.

Son ejes independientes de la apertura: un modelo de pesos abiertos también puede utilizarse mediante una API en la nube.

Local no equivale a privacidad automática: comprueba si el programa usa modelos remotos, telemetría o herramientas externas. **Nube no describe una única política de datos:** revisa el servicio, el plan y su configuración. Para practicar, usa código y datos que puedas compartir.

1.12. Cuantización y memoria: ¿cabe en mi equipo?

La **cuantización** representa los pesos con menor precisión para reducir su tamaño. Puede facilitar el despliegue y, según el hardware y el motor, acelerar la ejecución. El efecto sobre la calidad depende del modelo y del método.

Una aproximación para entender órdenes de magnitud es:

Memoria de los pesos $\approx$ parámetros totales $\times$ bits por parámetro $\div$ 8.

Ejemplo hipotético	Solo pesos a 4 bits, en GB decimales
4B	2 GB
8B	4 GB
27B	13,5 GB
120B	60 GB

Estos valores no son requisitos de equipo. Faltan los metadatos de cuantización, componentes a otra precisión, la memoria de trabajo, el sistema operativo y la caché del contexto. Esta última puede crecer con la longitud de la conversación; su tamaño depende también de la arquitectura. Consulta el archivo y la configuración que vas a ejecutar. [Cuantización en Transformers](#), [caché de contexto](#).

1.13. Actualidad, búsqueda y documentación

La fecha de conocimiento indica hasta cuándo llega parte de la información aprendida; no garantiza que el modelo conozca todo lo anterior ni las novedades posteriores.

La búsqueda web o la recuperación de documentos (habitualmente llamada **RAG**) aportan información externa durante la tarea. No son lo mismo que volver a entrenar el modelo. El **fine-tuning** (ajuste fino) modifica sus parámetros para adaptar comportamientos; no suele ser la primera solución para consultar documentación que cambia cada semana.

Aplicación práctica: cuando uses una biblioteca reciente, aporta la versión y su documentación oficial. Pide que las decisiones se apoyen en esa documentación y verifica que los enlaces existen y respaldan lo afirmado.

1.14. El coste real: modelo, aplicación y resultado

Con una **suscripción** pagas el acceso a una aplicación con sus condiciones y límites. Una **API** suele facturar por consumo. No compares una cuota mensual con un precio por millón de tokens como si fueran la misma unidad.

Al presupuestar una API, considera entrada, salida, razonamiento facturable, caché, herramientas, reintentos y posibles tarifas de contexto largo. Al usar un modelo local, considera el equipo, la energía y el mantenimiento.

La medida que más interesa es **coste por tarea correctamente resuelta**, incluyendo el tiempo de revisión. Los precios de esta guía sirven para entender las diferencias, no para prometer cuánto consumirá tu proyecto.

2. Cómo leer Artificial Analysis y LLM Stats

Estas son las dos principales fuentes de información y rankings sobre modelos, aunque sus puntuaciones no son intercambiables. Ten en cuenta que algunas cifras que aparecen en estos agregadores no han sido medidas directamente por sus responsables.

Fuente	Qué aporta	Cómo utilizarla
Artificial Analysis	https://artificialanalysis.ai - Evaluaciones propias, índices de capacidad y mediciones de coste, velocidad y latencia.	Revisa la versión del índice, el esfuerzo de razonamiento y el proveedor. Consulta también las pruebas específicas de programación y contexto.
LLM Stats	https://llm-stats.com - Catálogo amplio, tarifas, resultados de benchmarks y evaluaciones de preferencias. Combina evidencia propia y externa identificada.	Comprueba el origen de cada resultado. Su metodología pública es de alto nivel y no permite reproducir todos los detalles de su puntuación compuesta.

Fuentes: [metodología de Artificial Analysis](#), [metodología de LLM Stats](#), [construcción de LLM Stats Score](#).

Una fotografía actual, no un ranking permanente

Esta tabla utiliza **la información de Artificial Analysis de septiembre de 2026**. Es una muestra de modelos destacados en ese artículo, no una clasificación definitiva (lo habitual es que cambie con los lanzamientos de nuevos modelos, de ahí que sean tan importantes los fundamentos). Todos los valores de esta columna pertenecen a esa misma versión.

Modelo y configuración evaluada	Artificial Analysis Intelligence Index
Claude Fable 5.1, esfuerzo máximo y sustitución automática predeterminada (fallback)	53
GPT-6 Astra, esfuerzo máximo	53
Claude Opus 5, esfuerzo máximo	51
Claude Fable 5, con fallback	50
Muse Spark 1.3, esfuerzo máximo	48
GPT-5.6 Sol, esfuerzo máximo	47

Son **puntos de un índice**, no porcentajes de acierto en tu proyecto. El *fallback* significa que determinadas peticiones pueden ser atendidas por otro modelo; forma parte de la configuración evaluada. [Resultados de AA v4.3](#), [detalle de la evaluación de Fable 5.1](#).

El mismo artículo sitúa a **GLM-5.3 y Kimi K3** al frente de los modelos de pesos abiertos en su índice. Es una razón para incluirlos entre los candidatos, no para concluir que son pequeños o que convienen a cualquier tarea. [Resultados de AA v4.3](#).

En la consulta de **LLM Stats del 9 de septiembre**, el índice general mostraba Astra con **60,3**, Fable 5.1 con **56,3** y un valor de **55,1** para Sol, Muse Spark 1.3 y Opus 5. Es otra metodología: no debes promediar esos valores con los de AA ni interpretar su diferencia como una contradicción directa. [Tabla de LLM Stats](#).

Por qué importa la fecha: el artículo de Artificial Analysis sobre Astra del 3 de septiembre utilizaba el índice v4.1.1 y publicaba otra puntuación. El índice cambió después. Copiar números de artículos de lanzamiento y mezclarlos con fichas actuales produce una comparativa engañosa. [Evaluación de Astra del 3 de septiembre](#). Ten mucho cuidado.

Para programación, mira la prueba que se parece a tu trabajo

- **Programación de funciones y problemas:** indica cómo resuelve tareas de código delimitadas; no equivale a mantener una aplicación.
- **SWE-bench:** evalúa la resolución de incidencias en repositorios. Su conjunto Verified contiene 500 casos revisados por personas; no cubre todos los lenguajes ni todas las dimensiones de desarrollar software. [SWE-bench oficial](#).
- **Pruebas de agentes y terminal:** incluyen acciones sobre un entorno. Sus resultados dependen también de las herramientas, el presupuesto y la configuración del agente.
- **Arenas de preferencia:** muestran qué respuestas prefieren los evaluadores. Una solución atractiva todavía necesita comprobaciones de corrección y mantenibilidad.

Para comparar, anota **versión, herramientas, esfuerzo, número de intentos y presupuesto**. Una diferencia pequeña, especialmente con intervalos de incertidumbre solapados, no justifica proclamar un ganador.

3. Los cinco bloques de modelos

Los bloques organizan decisiones. **No son contenedores exclusivos:** un modelo económico puede razonar bien y uno de pesos abiertos puede servir para trabajo diario mediante una API.

Bloque	Cuándo empezar por él	Qué sacrificas o debes comprobar
A. Alta exigencia	Fallos difíciles, decisiones complejas, tareas largas.	Coste y espera; justificar que la mejora compensa.
B. Trabajo diario	Aprender, implementar, revisar y analizar documentos.	Probar sus límites antes de delegar cambios grandes.
C. Volumen y presupuesto ajustado	Muchas consultas, transformaciones y automatizaciones delimitadas.	Vigilar errores repetidos y coste de reintentos.
D. Pesos abiertos de gran capacidad	Control de despliegue o alternativas potentes por API.	Tamaño, licencia y diferencias entre pesos y servicio.
E. Uso local	Experimentar, trabajar con datos propios y aprender inferencia.	Memoria, compatibilidad y calidad en la tarea concreta.

Cómo leer las siguientes tablas

Precios: USD por un millón de tokens, **entrada sin caché / salida**. Son tarifas de API estándar del proveedor indicado; no cuotas de suscripción. No incluyen impuestos, herramientas, almacenamiento, modos rápidos, lotes ni descuentos de caché. Las excepciones y promociones se indican expresamente.

Contexto: K significa miles y M, millones de tokens; se redondea para facilitar la lectura. OpenAI publica una ventana conjunta y Google un límite de entrada: no interpretes ambas cifras como espacio íntegro para tus archivos. La aplicación puede admitir menos que la API.

Modalidades: T = texto, I = imágenes, A = audio, V = vídeo. Se indican las entradas documentadas para el servicio o modelo citado. **La salida de todos los modelos de estas tablas es texto.**

Entender vídeo no implica generarlo.

Las columnas de utilidad y límites contienen **orientación editorial para elegir qué probar**, apoyada en las fichas enlazadas. No son resultados de una competición propia.

A. Alta exigencia: cuando una mejora de capacidad puede compensar

Para: investigar un fallo con múltiples causas, revisar una arquitectura con restricciones o completar tareas complejas con herramientas.

Modelo	Contexto; entrada	API: entrada / salida
GPT-6 Astra · OpenAI	1,05M; T/I	10 / 50
Claude Fable 5.1 · Anthropic	1M; T/I	10 / 50
Claude Opus 5 · Anthropic	1M; T/I	5 / 25
Gemini 3.1 Pro Preview · Google	≈1,05M de entrada; T/I/A/V	2 / 12 hasta 200K de entrada

GPT-6 Astra · OpenAI

Por qué probarlo: Candidato para razonamiento y trabajo agéntico exigente; empate en la muestra AA v4.3 anterior. **Límite que debes tener presente:** Tarifa alta; recargo por contexto largo. Sus capacidades de agente requieren herramientas. [Ficha](#).

Claude Fable 5.1 · Anthropic

Por qué probarlo: Candidato para problemas exigentes y flujos largos; empate en esa muestra AA. **Límite que debes tener presente:** Razonamiento siempre activo y posibles sustituciones de modelo en ciertas solicitudes. [Ficha](#).

Claude Opus 5 · Anthropic

Por qué probarlo: Opción de alta capacidad con la mitad de tarifa por token que Fable en entrada y salida estándar. **Límite que debes tener presente:** Sigue siendo una tarifa elevada para tareas rutinarias; hay que medir el gasto completo. [Ficha](#).

Gemini 3.1 Pro Preview · Google

Por qué probarlo: Candidato para razonamiento sobre material multimodal. **Límite que debes tener presente:** Está en preview; por encima de 200K pasa a **4 / 18**. El nombre Pro no prueba superioridad frente a un Flash posterior. [Ficha](#), [tarifas](#).

Criterio de uso: antes de escalar, comprueba que has aportado el error, las restricciones y el contexto relevante. Cambiar de modelo no arregla una especificación ambigua.

Condición OpenAI: Astra aplica, por encima de 272K tokens de entrada, el doble de tarifa de entrada y caché y 1,5 veces la de salida a la petición completa. Los límites de uso de una aplicación o plan son otra cuestión. [Tarificación de Astra](#).

B. Trabajo diario: el punto de partida para comparar

Para: explicar conceptos, implementar funciones, revisar cambios pequeños, redactar documentación y analizar material de un proyecto.

Modelo	Contexto; entrada	API: entrada / salida
GPT-5.6 Sol · OpenAI	1,05M; T/I	4 / 20
GPT-5.6 Terra · OpenAI	1,05M; T/I	2 / 12
Claude Sonnet 5 · Anthropic	1M; T/I	2 / 10
Gemini 3.8 Flash · Google	≈1,05M de entrada; T/I/A/V	0,75 / 3,75, promoción
Muse Spark 1.3 · Meta	1M en Vercel; T/I, comprensión de V documentada	1,25 / 4,25, vía Vercel AI Gateway
Grok 4.6 · xAI	500K; T/I	2 / 6

GPT-5.6 Sol · OpenAI

Por qué probarlo: Escalón de capacidad para proyectos y problemas que se resisten a opciones económicas. **Límite que debes tener presente:** Más caro por token que Terra; precio promocional anunciado al menos hasta el 21/11/2026. [Ficha](#).

GPT-5.6 Terra · OpenAI

Por qué probarlo: Candidato equilibrado para iterar sobre código con razonamiento configurable. **Límite que debes tener presente:** La misma ventana que Sol no implica la misma capacidad. [Ficha](#).

Claude Sonnet 5 · Anthropic

Por qué probarlo: Candidato para implementar y revisar con menor tarifa que Opus y Fable. **Límite que debes tener presente:** Los cambios amplios siguen necesitando pruebas y revisión. [Ficha](#).

Gemini 3.8 Flash · Google

Por qué probarlo: Opción estable para combinar programación, documentación y entradas multimodales. **Límite que debes tener presente:** Precio promocional hasta el 31/12/2026; después se anuncia 1,50 / 7,50. [Ficha](#), [tarifas](#).

Muse Spark 1.3 · Meta

Por qué probarlo: Candidato para comparar trabajo agéntico y análisis visual a una tarifa inferior a los modelos del bloque A. **Límite que debes tener presente:** Pesos todavía privados. El precio es de este proveedor; el acceso y el contexto dependen de la integración. [Meta](#), [Vercel](#).

Grok 4.6 · xAI

Por qué probarlo: Alternativa para razonamiento y uso de herramientas, incluida búsqueda si la integración la ofrece. **Límite que debes tener presente:** Desde 200K de entrada la tabla de precios aplica 4 / 12; las herramientas pueden añadir cargos. [Ficha](#), [tarifas](#). Sol y Terra aplican también recargo a la petición completa por encima de 272K de entrada: doble en entrada y 1,5 veces en salida. [Sol](#), [Terra](#).

Si estás empezando: selecciona dos candidatos de este bloque o del siguiente que puedas usar con facilidad. Comprueba si explican bien tu código, respetan el alcance y ayudan a verificarlo. No necesitas contratar todos.

Ejemplo de por qué importa la aplicación: Cursor documenta 300K de contexto ordinario para Muse Spark 1.3 y 1M en Max Mode. El nombre del modelo no basta para saber cuánto contexto tienes realmente.

[Documentación de Cursor.](#)

C. Volumen y presupuesto ajustado: tareas pequeñas que se repiten

Para: clasificar incidencias, extraer campos, transformar datos, preparar borradores, explicar fragmentos y ejecutar automatizaciones acotadas.

Modelo/proveedor	Contexto; entrada	API: entrada / salida
GPT-5.6 Luna · OpenAI	1,05M; T/I	0,20 / 1,20
Qwen3.8-Flash · Qwen Cloud	1M; T/I/V	0,15 / 0,47
GLM-5.3-Flash · Z.ai	1M; T/I/V	0,15 / 0,50, tarifa normal
DeepSeek V4 Flash · DeepSeek	1M; T	0,22 / 0,66 valle; 0,44 / 1,32 punta
Mistral Small 4 · Mistral	256K; T/I	0,15 / 0,60

GPT-5.6 Luna · OpenAI

Por qué probarlo: Escalón económico con razonamiento configurable y herramientas. **Límite que debes tener presente:** No hereda la capacidad de Astra por compartir una ventana similar; tiene recargo por contexto largo. [Ficha.](#)

Qwen3.8-Flash · Qwen Cloud

Por qué probarlo: Candidato para automatizaciones y tareas de código con entrada visual. **Límite que debes tener presente:** La API incorpora funciones diferentes del checkpoint abierto Flash-Next. [Ficha y tarifas.](#)

GLM-5.3-Flash · Z.ai

Por qué probarlo: Candidato económico con visión, herramientas y pesos publicados. **Límite que debes tener presente:** Razonamiento siempre activo; 320B totales, por lo que «Flash» no significa pequeño para local. [Ficha](#), [tarifas](#).

DeepSeek V4 Flash · DeepSeek

Por qué probarlo: Candidato para código y automatizaciones textuales, con o sin razonamiento. **Límite que debes tener presente:** Esta variante no tiene visión; el precio depende del horario. [Modelos](#), [tarifas](#).

Mistral Small 4 · Mistral

Por qué probarlo: Alternativa multimodal con pesos Apache 2.0 y opciones de razonamiento. **Límite que debes tener presente:** 119B totales y 6,5B activos: requiere dimensionar el despliegue propio. [Ficha](#), [tarifas](#).

Dos alternativas dentro de otros ecosistemas: Claude Haiku 4.5 sigue activo a **1 / 5**, con 200K de contexto y entrada de texto e imagen; Gemini 3.5 Flash-Lite ofrece entrada multimodal y ≈1,05M de entrada a **0,30 / 2,50**. Pueden encajar si ya usas esos servicios, aunque sus nombres no garantizan el menor coste ni la mayor velocidad entre compañías. [Haiku](#), [Flash-Lite](#), [tarifas Google](#).

Condiciones que evitan comparaciones engañosas:

- Para GLM-5.3-Flash se muestra la tarifa normal. La promoción de lanzamiento al 50% termina el 9/09/2026 a las 24:00 UTC+8; no se usa como precio permanente de la guía.
- DeepSeek considera horas punta de lunes a viernes las franjas **01:00–04:00 y 06:00–10:00 UTC**; fuera de ellas aplica la tarifa valle. [Precios vigentes](#).
- Luna duplica entrada y multiplica salida por 1,5 por encima de 272K de entrada en la petición completa. [Condiciones](#).

Criterio de uso: una tarifa baja resulta útil si el modelo cumple el contrato de la tarea. Para extraer JSON, valida el esquema y los valores. Si un error se repite cientos de veces, el ahorro inicial puede desaparecer.

D. Pesos abiertos de gran capacidad: control y alternativas por API

Para: comparar opciones potentes fuera de los modelos privados, adaptar un despliegue o alojar un modelo con infraestructura adecuada.

Un estudiante puede probarlos por API sin descargarlos.

Modelo y servicio de referencia	Contexto y entrada del servicio	API: entrada / salida
GLM-5.3 · Z.ai	1M; T	1,40 / 4,40
Kimi K3 · Moonshot	≈1,05M; T/I, comprensión de V	3 / 15
Kimi K2.7 Code · Moonshot	256K; T/I	0,95 / 4
Qwen3.8-Max · Qwen Cloud	1M; T/I/V en API	2 / 6
DeepSeek V4 Pro · DeepSeek	1M; T	0,66 / 1,98 valle; 1,32 / 3,96 punta
Mistral Medium 3.5 · Mistral	256K; T/I	1,50 / 7,50

GLM-5.3 · Z.ai

Pesos publicados: Licencia propia GLM-5.3; ≈753B según recuento de tensores. Activos no confirmados aquí. **Principal ventaja y límite práctico:** Candidato de gran capacidad textual; aparece destacado entre abiertos en AA. Razonamiento siempre activo, sin visión y despliegue voluminoso. [API](#), [pesos](#), [precio](#).

Kimi K3 · Moonshot

Pesos publicados: Kimi K3 License; 2,8T totales / 104B activos. **Principal ventaja y límite práctico:** Candidato para tareas complejas con contexto y visión. Razonamiento siempre activo; apertura no implica bajo precio ni viabilidad en portátil. [Pesos](#), [precio](#).

Kimi K2.7 Code · Moonshot

Pesos publicados: Modified MIT; 1T / 32B activos. **Principal ventaja y límite práctico:** Especialización en programación con menor tarifa que K3. Menor contexto y sigue siendo un modelo enorme. [Pesos](#), [tarifas](#).

Qwen3.8-Max · Qwen Cloud

Pesos publicados: Qwen3.8-2.4T-A95B, licencia propia; 2,4T / 95B activos.

Principal ventaja y límite práctico: Candidato para escalar desde Flash.

El checkpoint abierto citado es textual; la API añade visión. [API y precio](#), [pesos](#).

DeepSeek V4 Pro · DeepSeek

Pesos publicados: MIT; 1,6T / 49B activos en los pesos publicados.

Principal ventaja y límite práctico: Candidato para problemas que se resisten a Flash a una tarifa contenida. Sin visión; hay que distinguir revisión de API y checkpoint. [API/precio](#), [pesos](#).

Mistral Medium 3.5 · Mistral

Pesos publicados: Modified MIT; 128B densos.

Principal ventaja y límite práctico: Alternativa multimodal para API o infraestructura propia. Menor ventana que varios competidores y licencia con condiciones específicas. [Ficha](#), [tarifas](#).

Tres detalles que merece la pena recordar:

1. GLM-5.3 usa una licencia propia; GLM-5.3-Flash usa MIT. Kimi K3 y K2.7 Code tampoco comparten exactamente la misma licencia. Revisa el texto de la versión que vayas a distribuir o integrar.

2. Qwen3.8-Flash-Next declara 125B de parámetros con 6B activos, más 51B de embeddings y 4B de otro componente de predicción: alrededor de **180B almacenados**. Ilustra por qué 6B activos no equivalen a una descarga de 6B. Su licencia impone condiciones comerciales específicas, incluidas algunas clases de asistentes. [Tarjeta](#), [licencia](#).

3. La API de DeepSeek identifica V4 Flash con la revisión **0731** y V4 Pro con **0813**. Las tarjetas de pesos consultadas conservan la etiqueta preview; aquí no se presupone que sean idénticas a esas revisiones del servicio. [API](#), [pesos Flash](#), [pesos Pro](#).

Criterio de uso: elige pesos abiertos cuando su licencia, control o despliegue aporten valor a tu proyecto. Si sólo necesitas resolver ejercicios, también puedes comparar sus APIs con las opciones privadas usando las mismas tareas.

E. Uso local: empieza por algo que puedas ejecutar y evaluar

Para: aprender cómo funciona la inferencia, experimentar con herramientas y procesar datos en tu equipo. No hace falta elegir el modelo más reciente si uno anterior cumple mejor tus restricciones.

Modelo	Tamaño publicado	Contexto nativo; entrada	Licencia
Qwen3.5-4B	4B del componente de lenguaje más codificador visual	262K; T/I/V	Apache 2.0
Gemma 4 E4B · Google	4,5B efectivos; 8B con embeddings, más componentes multimodales documentados	128K; T/I/A y V por fotogramas	Apache 2.0
Ministral 3 8B Instruct 2512 · Mistral	8,4B de lenguaje + 0,4B de visión	256K; T/I	Apache 2.0
gpt-oss-20b · OpenAI	≈21B totales / 3,6B activos	131K; T	Apache 2.0
Qwen3.8-27B	27B de lenguaje, denso, más componente visual	262K; T/I/V	Apache 2.0

Qwen3.5-4B

Para qué probarlo y qué comprobar: Primeras pruebas de código y capturas. El nombre 4B no incluye todo el componente visual. [Ficha](#).

Gemma 4 E4B · Google

Para qué probarlo y qué comprobar: Audio de hasta 30 segundos y vídeo de hasta 60 segundos a 1 fps según la ficha. E4B no significa que todos sus pesos sumen 4B; revisa el soporte del motor. [Ficha](#).

Ministral 3 8B Instruct 2512 · Mistral

Para qué probarlo y qué comprobar: Ejercicios y comprensión visual con una variante local compatible. El contexto máximo no es una configuración ligera. [Ficha](#).

gpt-oss-20b · OpenAI

Para qué probarlo y qué comprobar: Aprender razonamiento y conexión de herramientas en local. Sólo texto y mayor almacenamiento que un modelo de 4B. [Ficha](#).

Qwen3.8-27B

Para qué probarlo y qué comprobar: Candidato más exigente para quien dispone de memoria suficiente. No deducir que iguala a un modelo de frontera en todas las tareas. [Ficha](#).

Algunas fichas Qwen documentan extensiones de contexto superiores al nativo. Requieren configuración y recursos: la tabla utiliza el límite nativo para no prometer el máximo ampliado como experiencia de un portátil.

Cómo empezar: selecciona un modelo compacto, una cuantización y un motor que soporte esa versión. Usa una ventana moderada, comprueba la memoria ocupada y prueba unas pocas tareas reales. Sube de tamaño cuando identifiques una mejora necesaria y tengas recursos para ello.

No se asigna una «RAM mínima universal» porque cambia con el archivo, la precisión, las modalidades y el contexto. La tabla de memoria de los fundamentos permite entender la escala; la configuración concreta determina si funciona de forma útil.

Una especialización adicional: autocompletado de código

Completar unas líneas dentro del editor exige poca espera y puede usar modelos especializados en **FIM (rellenar código entre un prefijo y un sufijo)**. No es la misma tarea que pedir a un agente que revise y modifique varios archivos.

Como ejemplo vigente, Mistral documenta **Codestral codestral-250B**, 128K de contexto y tarifa de **0,30 / 0,90**. Para este uso, compara la integración con el editor, la latencia y cuántas sugerencias aceptas sin corregir. [Ficha de Codestral, tarifas](#).

4. Qué elegir según la tarea que tengas delante

Esta tabla propone **puntos de partida**, no ganadores universales (para gustos, hay colores). Los modelos de cada fila se apoyan en las fichas anteriores; la elección final depende de tu evaluación y del acceso que tengas.

Tu tarea	Candidatos para empezar	Qué debe decidir
Aprender un concepto o entender una función	Gemini 3.8 Flash, Sonnet 5 o Terra; si priorizas coste, Qwen3.8-Flash	Explica a tu nivel, plantea preguntas y permite comprobar lo aprendido.
Implementar una función o corregir un fallo acotado	Sonnet 5, Terra, GLM-5.3-Flash o DeepSeek V4 Flash	Cumple el contrato, respeta el proyecto y supera los casos relevantes.
Resolver un fallo difícil o decidir una arquitectura	Astra, Fable 5.1, Opus 5, GLM-5.3 o Kimi K3	Encuentra causas comprobables, compara alternativas y justifica los compromisos.
Trabajar sobre un repositorio durante varios pasos	Sol, Opus 5, Muse Spark 1.3 o Kimi K2.7 Code, en una integración con herramientas	Navega el proyecto, acota cambios, ejecuta comprobaciones y recupera errores. Escala si hace falta.
Convertir una captura en una propuesta de interfaz	Gemini 3.8 Flash, Muse Spark 1.3 o Qwen3.8-Flash	Comprende la captura y el resultado funciona visualmente en el navegador.
Analizar un vídeo o audio de una clase	Gemini 3.8 Flash; Gemma 4 E4B para fragmentos locales de audio de hasta 30 segundos	Admite la modalidad en tu aplicación y permite verificar fragmentos y transcripción.
Analizar CSV, métricas o registros	Un modelo del bloque B con una herramienta de cálculo; uno del C para extracción delimitada	Ejecuta cálculos reproducibles y mantiene los datos fieles al original.
Escribir documentación, un artículo o un guion técnico	Sonnet 5, Gemini 3.8 Flash o Qwen3.8-Flash	Exactitud, claridad, fuentes y adaptación al público; no sólo estilo convincente.
Hacer cientos de transformaciones o clasificaciones	Luna, Qwen3.8-Flash, GLM-5.3-Flash, Mistral Small 4 o DeepSeek V4 Flash	Coste por resultado válido, cumplimiento del formato y tratamiento de errores.
Trabajar en tu equipo con un modelo descargado	Qwen3.5-4B, Gemma 4 E4B o Ministral 3 8B; luego valorar modelos mayores	Compatibilidad, memoria, privacidad del flujo completo y calidad suficiente.

Para **crear imágenes, vídeo o voz**, consulta comparativas específicas de generación. Para **buscar documentos por similitud**, evalúa modelos de *embeddings* y, si procede, de reordenación de resultados. Un LLM generalista puede coordinar esas tareas, pero su puntuación en programación no determina la calidad de esos componentes.

Una secuencia sencilla para decidir

- 1. Define qué significa terminar bien.** Por ejemplo: «corregir el error sin cambiar la API y demostrarlo con una prueba».
- 2. Filtra por requisitos obligatorios.** Modalidad, privacidad, licencia, contexto suficiente y acceso desde tu herramienta.
- 3. Elige dos candidatos razonables.** Uno económico o equilibrado y otro con más capacidad si la tarea lo requiere.
- 4. Prueba con el mismo material.** Compara corrección, intervención, tiempo y consumo.
- 5. Quédate con el que cumple a menor coste total.** Si empatan, prioriza facilidad de uso y revisión.

Antes de aumentar capacidad, revisa el contexto y la especificación. Si el problema persiste, aumenta el esfuerzo de razonamiento o cambia de modelo; modificar ambas cosas a la vez dificulta saber qué produjo la mejora.

5. Entender cuánto cuesta realmente

Para una petición de texto sencilla, sin caché ni herramientas adicionales:

Coste = (tokens de entrada ÷ 1.000.000 × tarifa de entrada) + (tokens de salida facturables ÷ 1.000.000 × tarifa de salida).

La salida facturable debe incluir el razonamiento si el proveedor lo cobra ahí. No lo sumes dos veces si ya está incluido en el contador de salida. Con caché o tramos de contexto, aplica sus condiciones por separado.

Ejemplo calculado: el mismo volumen facturable

Supongamos **100.000 tokens de entrada y 10.000 de salida facturable**. No hay caché, herramientas adicionales ni modo rápido. Estos cálculos propios usan las tarifas de las tablas anteriores; no son una medición de cuánto necesita cada modelo para resolver un ejercicio.

Modelo / condición	Entrada	Salida	Total de la petición hipotética
GPT-6 Astra, estándar	1,0000 USD	0,5000 USD	1,5000 USD
Claude Sonnet 5, estándar	0,2000 USD	0,1000 USD	0,3000 USD
Gemini 3.8 Flash, promoción 2026	0,0750 USD	0,0375 USD	0,1125 USD
DeepSeek V4 Flash, valle	0,0220 USD	0,0066 USD	0,0286 USD
GLM-5.3-Flash, tarifa normal	0,0150 USD	0,0050 USD	0,0200 USD
Qwen3.8-Flash, tarifa publicada	0,0150 USD	0,0047 USD	0,0197 USD

Fuentes de tarifas: [OpenAI](#), [Anthropic](#), [Google](#), [DeepSeek](#), [Z.ai](#), [Qwen Cloud](#).

Esta tabla compara tarifas con cantidades fijadas, no calidad ni coste por tarea. En la práctica, cada modelo puede tokenizar de forma diferente, generar más razonamiento, usar herramientas distintas o necesitar más intentos. Una sesión con un agente puede contener muchas peticiones como estas.

Hay evidencia de que la diferencia importa: en su evaluación de agentes de programación del **3 de septiembre**, AA observó que Astra, pese a tener mayor tarifa por token que Sol, costaba aproximadamente lo mismo por tarea a esfuerzo máximo gracias a un menor consumo. Ese resultado corresponde a aquella evaluación y configuración; no se extrapola a todo uso ni se mezcla con el índice v4.3. [Evaluación publicada por AA](#).

Cuatro formas de gastar con criterio

- **Acota el contexto:** manda lo necesario y amplíalo según el problema.
- **Ajusta la salida:** pide el cambio y su explicación útil, evitando documentos innecesarios.
- **Usa caché o lotes cuando encajen:** comprueba las tarifas de lectura, escritura y almacenamiento; no todos funcionan igual.
- **Controla el flujo:** establece presupuestos, límites de pasos y criterios para detener reintentos que no avanzan.

Si utilizas una suscripción, observa también sus límites de uso y cuánto consume el modo elegido. Estos importes de API no permiten deducir cuántas tareas incluye un plan mensual.

6. Tu propia comparativa: un ejercicio que sí te enseña a elegir

No necesitas un laboratorio para obtener una primera señal útil. **Este ejercicio está propuesto para el alumno; no se ha ejecutado como parte de esta guía.**

Escoge dos modelos y conserva la misma versión del proyecto, instrucciones, archivos y herramientas. Si cambias también de aplicación, registra que estás comparando el conjunto. Usa un presupuesto y un número de intentos equivalentes.

Prueba	Qué observar
Explicar una función que ya entiendes	Detecta errores conceptuales, usa ejemplos correctos y adapta la explicación a tu nivel.
Corregir un fallo reproducible	Identifica la causa; una prueba falla antes y pasa después; no modifica código sin relación.
Implementar una función con contrato claro	Cumple requisitos, casos límite y dependencias permitidas.
Revisar un cambio con un error conocido	Encuentra el error real, lo localiza y evita señalar problemas inventados.
Usar documentación de una versión concreta	Emplea APIs existentes y distingue lo documentado de lo supuesto.

Un encargo común para probar implementación

Puedes entregar este mismo encargo a ambos modelos:

Estoy aprendiendo Python. Implementa `normalizar_etiquetas(etiquetas)`, que recibe una lista de cadenas y devuelve una nueva lista de cadenas.

Requisitos:

- Elimina espacios al principio y al final de cada etiqueta.
- Convierte las etiquetas a minúsculas.
- Descarta las etiquetas vacías después de normalizarlas.
- Elimina duplicados conservando el orden de la primera aparición.
- No modifiques la lista de entrada ni uses dependencias externas.
- Puedes asumir que todos los elementos de entrada son cadenas.

Incluye casos para lista vacía, espacios, mayúsculas y duplicados.

Explica la complejidad temporal esperada y por qué se conserva el orden.

Si puedes ejecutar las pruebas, muestra el resultado. Si no puedes, indica que sólo has propuesto las pruebas.

Comprueba tú el resultado. Añade después un caso que no aparecía en los ejemplos. Para evaluar tutoría, puedes transformar el encargo: pide primero una pista y una pregunta que te ayude a construir la solución, sin mostrarla completa.

Registro mínimo de resultados

Resultado e intervención

Tarea	Modelo, versión y esfuerzo	Resultado	Ayuda e intentos
Explicación	Completar	Correcto / con errores	Completar
Corrección del fallo	Completar	Resuelto / con ayuda / no resuelto	Completar
Implementación	Completar	Resuelto / con ayuda / no resuelto	Completar
Revisión	Completar	Hallazgo válido / falso positivo / omitido	Completar
Documentación	Completar	Verificado / no verificado	Completar

Tiempo, coste y comprensión

Tarea	Tiempo total	Coste observado	¿Puedo explicarlo?
Explicación	Completar	Completar	Sí / no
Corrección del fallo	Completar	Completar	Sí / no
Implementación	Completar	Completar	Sí / no
Revisión	Completar	Completar	Sí / no
Documentación	Completar	Completar	Sí / no

Repite las tareas donde el resultado sea dudoso. Una sola respuesta acertada es una señal débil; conserva los resultados para comparaciones posteriores. No conviertas cinco ejercicios personales en una afirmación sobre todos los programadores.

7. Errores frecuentes al elegir un modelo

Error	Mejor criterio
«Uso siempre el más potente».	Escalar cuando la tarea y los resultados justifiquen el gasto.
«Más parámetros significa mejor código».	Evaluar el resultado y usar el tamaño para dimensionar recursos.
«Tiene un millón de contexto: le paso todo».	Aportar contexto relevante y comprobar qué restricciones retiene.
«Es Flash o Small: cabrá en mi portátil».	Mirar parámetros totales, archivo, precisión y memoria durante el uso.
«Es de una marca abierta: puedo usarlo sin condiciones».	Leer la licencia del checkpoint concreto.
«Está en un ranking: cualquiera puede acceder».	Comprobar si es público, preview, experimental o de acceso restringido.
«La respuesta suena segura: estará bien».	Verificar documentación, ejecutar y revisar.
«Los tests pasan: el cambio ya es perfecto».	Comprobar cobertura relevante, requisitos y mantenibilidad.
«Más razonamiento siempre mejora».	Medir la mejora frente al tiempo y consumo adicional.
«El agente terminó: yo he aprendido».	Explicar la solución y modificarla después por tu cuenta.

8. Llegar al curso con una buena base

Para preparar «**Desarrollo con IA: El nuevo programador**», basta con una selección pequeña y un ejercicio real:

1. Elige una herramienta accesible y un modelo del bloque B o C.
2. Lleva un pequeño proyecto que puedas ejecutar y entender.
3. Guarda un fallo o una mejora concreta para trabajar con la IA.
4. Conserva la conversación, los cambios y las comprobaciones.
5. Anota qué resolviste, qué aprendiste y qué tuviste que corregir.

Tu objetivo es desarrollar un proceso que puedas repetir: **comprender → pedir → revisar → ejecutar → comprobar → aprender**.

Los fundamentos te permiten formular el problema. La IA amplía lo que puedes probar y construir. Tu criterio decide qué aceptar, qué cambiar y qué verificar.

Continúa aprendiendo con los **recursos exclusivos que estoy compartiendo en la comunidad privada de WhatsApp**.

[Entra haciendo clic aquí](#) y llega a los directos del Curso de Desarrollo con IA: El Nuevo Programador, con una base más sólida.

ACTIVA LA CAMPANITA EN YOUTUBE

***PARA QUE TE AVISE CUANDO
COMIENCE LA CLASE #1 DEL CURSO***



***ACCEDE AQUÍ A LA
PRIMERA CLASE***

BIG school